Governance Architecture for AI Coding Agent Harnesses

First Author[1], Second Author[2], Third Author[2], Fourth Author[1] and Fifth Author[1]
[1]Pragnition Labs, [*]Equal Contribution, [1]Affiliation 1, [2]Affiliation 2

AI coding agents now generate over 50% of committed code on major platforms, yet architectural governance remains designed for human-speed development. Analysis of 211 million changed lines shows an 8× increase in code duplication and a 60% collapse in refactoring activity following mass AI adoption. We present a formal framework for governance architecture in AI coding harnesses, grounding it in three mathematical pillars: (1) information-theoretic bounds on theory externalization, connecting Naur's "programming as theory building" to rate-distortion theory and establishing that tacit knowledge components have divergent description length; (2) a control-theoretic model of multi-granularity governance as a cascade control system, with Nyquist-like sampling requirements and a transcritical bifurcation separating self-correcting regimes from coherence collapse; and (3) survival analysis of decision validity with competing risks, providing empirically calibrated decay rates for architectural decisions across domains. We prove a *governance capacity bound*: when the rate of architectural drift exceeds governance channel capacity, no governance scheme can prevent unbounded divergence, regardless of sophistication. We formalize the "ratchet" enforcement pattern as a closure operator on a lattice of constraints, prove its convergence, and identify the ossification conditions under which it becomes pathological. Five contrarian positions (governance as overhead, ADRs as documentation theater, Naur's unsolvability thesis, Conway's Law futility, ratchet rigidity) are engaged with empirical evidence rather than dismissed. The framework synthesizes results from information theory, control theory, survival analysis, and the tacit knowledge literature into a unified governance information budget that quantifies the fundamental tensions facing AI-accelerated software development.

1. Introduction

The velocity problem in AI-assisted software development is not a management challenge; it is an information-theoretic one. When AI agents modify codebases faster than humans can evaluate the implications, architectural coherence degrades not because of carelessness but because of a fundamental mismatch between the rate at which drift is injected and the rate at which governance can correct it.

The empirical evidence is stark. GitClear's analysis of 211 million changed lines (2020–2024) found that copy-pasted code rose from 8.3% to 12.3% of changes, code blocks with five or more duplicated lines increased approximately 8×, and refactoring dropped from 25% to under 10% of changed lines (GitClear, 2025). The DORA 2025 report confirmed that while individual task completion increased 21% with AI adoption, organizational-level throughput stayed flat and software delivery stability showed a *negative* correlation (Google Cloud, 2025). Sylvester (2026) calls this "context drift": the agent's working model diverges from the actual architecture with every generation.

The core tension is temporal. Human architects think in trajectories (where is this system heading?). AI agents think in snapshots (what does this prompt need right now?). Drift is the integral of that gap over time.

This paper presents a formal framework for governance architecture in AI coding harnesses. Our

contributions are:

1. An **information-theoretic formalization** of theory loss, connecting Naur's "programming as theory building" (Naur, 1985) to rate-distortion theory and establishing formal bounds on the extractability of program theory from execution logs (§3).
2. A **governance capacity bound** (Theorem 4.1): an analog of Shannon's channel coding theorem showing that when drift rate exceeds governance channel capacity, no governance scheme prevents unbounded divergence (§4).
3. A **cascade control model** of multi-granularity governance with stability analysis and Nyquist-like sampling requirements, including a transcritical bifurcation at the critical governance capacity (§5).
4. A **lattice-theoretic formalization** of the ratchet enforcement pattern as a closure operator, with convergence proofs and ossification conditions (§6).
5. A **competing-risks survival model** for decision validity with empirically calibrated decay rates across architectural domains (§7).
6. Honest engagement with five **contrarian positions**, evaluating what the evidence actually supports rather than constructing strawmen (§11).

We frame this as a theory and position paper. The formal results are novel in their *application* to governance architecture; the underlying mathematical machinery (rate-distortion theory, control theory, survival analysis, lattice theory) is established. Where we conjecture rather than prove, we say so explicitly.

2. Preliminaries and Definitions

2.1. Program Theory (Naur)

Naur (1985) argues that programming is an activity of theory building, where "theory" is drawn from Ryle (1949): the knowledge a person must have "in order not only to do certain things intelligently but also to explain them, to answer queries about them, to argue about them." Naur identifies three capabilities that resist documentation: (1) mapping between code and reality, (2) design justification ("the justification is and must always remain the programmer's direct, intuitive knowledge or estimate"), and (3) modification intelligence, which depends on recognizing similarities that "are not, and cannot be, expressed in terms of criteria."

Definition 2.1 (Program Theory)**.** *A* program theory T *is a random variable over a measurable space* $(\mathcal{T}, \Sigma_{\mathcal{T}})$ *representing the totality of a developer's (or agent's) mental model of a program, including the mapping from problem domain to code, design rationale, rejected alternatives, and the causal model of change propagation.*

2.2. Tacit Knowledge Taxonomy

Collins (2010) advances beyond Polanyi (1966) by taxonomizing tacit knowledge into three types with distinct externalization properties:

- **Relational tacit knowledge (RTK):** Knowledge that *could* be made explicit but has not been, due to social contingency. ADRs and governance tools operate effectively in this domain.
- **Somatic tacit knowledge (STK):** Knowledge residing in embodied practice, partially capturable through instrumentation but not transmissible through text.

- **Collective tacit knowledge (CTK):** Knowledge "embodied in society," acquired only through enculturation. Resistant to externalization by any known means.

Tsoukas (2003) critiques the assumption (from Nonaka and Takeuchi 1995) that tacit knowledge is simply "knowledge not yet articulated," arguing that tacit and explicit knowledge are two dimensions of *all* knowing, not two pools that can be converted back and forth.

2.3. Architectural State and Drift

Definition 2.2 (Architectural Drift)**.** *Let P denote the intended architecture (a distribution over architectural states $\mathcal{S}$) and Q_t the actual architecture at time t. The* architectural drift *is:*

$$\Delta(t) = D_{\mathrm{KL}}(P \,\|\, Q_t) = \sum_{s \in \mathcal{S}} P(s) \log \frac{P(s)}{Q_t(s)} \tag{1}$$

Drift is zero when intended and actual architectures coincide and grows as they diverge.

2.4. Reflexion Models

Murphy et al. (1995, 2001) introduced reflexion models for architectural conformance checking. Given a high-level model $HM = (E_H, R_H)$, a source model $SM = (E_S, R_S)$, and a mapping $M \subseteq E_S \times E_H$, the reflexion model classifies each high-level relationship as a *convergence* (intended and present), *divergence* (present but not intended), or *absence* (intended but not present).

3. Information-Theoretic Foundations

3.1. Theory Loss as a Rate-Distortion Problem

Definition 3.1 (Extraction Channel)**.** *An* extraction channel *is a conditional distribution $p(\hat{T} \mid T)$ representing the process of externalizing theory T into an artifact $\hat{T}$ (documentation, ADRs, structured logs). A* distortion function $d : \mathcal{T} \times \hat{\mathcal{T}} \to [0, \infty)$ *measures reconstruction loss, with $d(t, \hat{t}) = 0$ iff $\hat{t}$ perfectly reconstructs t.*

Theorem 3.1 (Theory Extraction Bound)**.** *The process of extracting program theory into documentation is a lossy source coding problem. For any extraction mechanism achieving average distortion D, the extraction must produce at least $R(D)$ bits of structured output, where:*

$$R(D) = \min_{p(\hat{T}|T) \colon \mathbb{E}[d(T,\hat{T})] \le D} I(T; \hat{T}) \tag{2}$$

is the rate-distortion function (Cover and Thomas, 2006; Shannon, 1959).

Proof. Direct application of Shannon's rate-distortion theorem. The extraction channel $p(\hat{T} \mid T)$ is a (possibly stochastic) encoder; the artifact $\hat{T}$ is the compressed representation; reconstruction is whatever a future reader infers from $\hat{T}$. □

The key claim, connecting Naur and Polanyi, is that program theory contains components for which this function diverges.

Conjecture 3.1 (Tacit Divergence). *Decompose the program theory as* $T = (T_{\text{explicit}}, T_{\text{tacit}})$. *For the tacit component:*

$$\lim_{D \to 0} R_{\text{tacit}}(D) = \infty \tag{3}$$

with $R_{\text{tacit}}(D) = \Omega(\log(1/D))$ *as* $D \to 0$. *In contrast,* $R_{\text{explicit}}(D)$ *remains finite for all* $D \geq 0$.

This conjecture formalizes Naur's observation that theory building cannot be reduced to documentation. Its epistemic status is that of a *philosophically motivated mathematical conjecture*: the divergence claim is supported by the structure of rate-distortion theory and the philosophical arguments of Naur, Polanyi, and Collins, but a rigorous proof would require specifying the source distribution and distortion measure for tacit knowledge, which is itself an open problem.

Concrete example. For a Gaussian source with squared-error distortion, $R(D) = \frac{1}{2}\log(\sigma^2/D)$, which diverges as $D \to 0$ but only logarithmically. Model design rationale as a mixture: 70% explicit component (finite alphabet, finite $R(D)$) and 30% tacit component (continuous source with variance σ_τ^2). Even under this simple model, the mixture rate-distortion function diverges as $D \to 0$ because the continuous component cannot be perfectly reconstructed with finite description length. For heavier-tailed sources (modeling the long tail of rare design judgments), the divergence rate exceeds logarithmic.

3.2. Incompressibility of Tacit Components

Theorem 3.2 (Incompressibility). *If* T_{tacit} *contains components that are Kolmogorov-random relative to any formal documentation language* $\mathcal{L}$, *then for those components* t_τ:

$$K(t_\tau) \geq |t_\tau| - O(1) \tag{4}$$

where K *denotes Kolmogorov complexity. Any documentation artifact is at least as long as the tacit knowledge it captures.*

Proof. By the incompressibility theorem from algorithmic information theory (Chaitin, 1974), a fraction $\geq 1 - 2^{-c}$ of strings of length n satisfy $K(x) \geq n - c$. If tacit components are drawn from a distribution placing non-negligible mass on incompressible strings (relative to $\mathcal{L}$), they cannot be compressed with high probability. □

Corollary 3.1 (Governance Implication). *No governance mechanism based on artifact inspection alone can detect the loss of tacit knowledge components with* $K(t_\tau) \geq |t_\tau| - O(1)$. *Governance must include mechanisms that probe the theory holder directly (code review, design critique, architectural challenge).*

3.3. Extraction Fidelity and the Double Bottleneck

AI coding agents produce verbose execution logs L (often 100K+ tokens). A governance system must extract structured knowledge (D, A, R) (decisions, assumptions, rejected alternatives). This forms a Markov chain: $T \to L \to (D, A, R) \to G$, where G represents governance actions.

Theorem 3.3 (Double Bottleneck). *By the data processing inequality, applied twice:*

$$I(T; G) \leq I(T; (D, A, R)) \leq I(T; L) \tag{5}$$

Each stage of the governance pipeline loses information. Governance fidelity is bounded above by log richness.

Corollary 3.2 (Log Richness Lower Bound). *For governance to achieve fidelity* $I(T; G) \geq F_{\min}$, *the agent log must satisfy* $I(T; L) \geq F_{\min}$. *Terse logs impose an upper bound on governance fidelity regardless of extraction sophistication.*

4. Governance Channel Capacity

Definition 4.1 (Governance Channel)**.** *Model governance as a discrete memoryless channel* $(X, p(Y \mid X), Y)$ *where* X *represents governance signals (review comments, CI checks, policy violations),* Y *represents resulting code modifications, and* $p(Y \mid X)$ *captures transmission noise (missed issues, misinterpreted feedback, false negatives).*

Definition 4.2 (Governance Channel Capacity)**.**

$$C_{\text{gov}} = \max_{p(X)} I(X; Y) \tag{6}$$

the maximum mutual information between governance signals and resulting changes, optimized over the distribution of governance actions.

Theorem 4.1 (Governance Capacity Bound)**.** *Let* R_{drift} *be the rate of architectural drift (bits per unit time). Let* C_{gov} *be the governance channel capacity. Then:*

1. *If* $R_{\text{drift}} < C_{\text{gov}}$*, there exists a governance coding scheme (combination of reviews, tests, policies, automated checks) maintaining architectural coherence with arbitrarily small residual drift.*
2. *If* $R_{\text{drift}} > C_{\text{gov}}$*,* ***no governance scheme*** *can prevent architectural drift from growing without bound.*

Proof. By structural analogy to Shannon's noisy channel coding theorem (Shannon, 1948). Part (1): when drift rate is below capacity, governance corrections can be encoded with sufficient redundancy. The "coding scheme" corresponds to layered governance (linting catches syntactic drift, CI tests catch behavioral drift, human review catches semantic drift, ADRs catch strategic drift). Part (2): by the converse of the channel coding theorem, when drift rate exceeds capacity, governance failure probability is bounded away from zero. □

Remark (Epistemic Status)**.** *Theorem 4.1 is a* structural analogy*, not a direct application of Shannon's theorem. The governance "channel" lacks the well-defined input/output alphabets and stationary transition probabilities of a discrete memoryless channel. The theorem's value is in design guidance: it formalizes the intuition that governance systems have finite corrective bandwidth and that exceeding it produces qualitatively different (unbounded) behavior. The analogy is rigorous in structure but the quantities* R_{drift} *and* C_{gov} *are not directly measurable in the way Shannon's channel capacity is for communication channels. We retain the theorem framing because the structural parallel is exact, while flagging that empirical instantiation requires defining concrete governance scenarios with measurable information rates.*

Proposition 4.1 (Capacity Decomposition)**.** *For a layered governance system with* L *independent layers, the combined capacity satisfies:*

$$C_{\text{gov}} \leq \sum_{\ell=1}^{L} C_\ell \tag{7}$$

with equality when layers operate on non-overlapping aspects. When layers overlap, the combined capacity is strictly less than the sum, but error probability decreases exponentially in the number of redundant layers.

Corollary 4.1 (AI Velocity Problem)**.** *When AI agents increase the modification rate* λ *while* C_{gov} *remains constant (human review bandwidth is fixed), the condition* $R_{\text{drift}} > C_{\text{gov}}$ *is eventually satisfied. AI-accelerated development inevitably overwhelms governance systems of fixed capacity.*

Corollary 4.2 (Automated Governance Necessity)**.** *Maintaining* $R_{\text{drift}} < C_{\text{gov}}$ *under increasing* λ *requires increasing* C_{gov} *commensurately. Since human review capacity is bounded, automated governance layers (fitness functions, architectural tests, invariant checkers) are* necessary*, not merely convenient.*

5. Multi-Granularity Governance

5.1. Three Governance Loops

A governance architecture operates at three temporal granularities, each corresponding to a loop in a cascade control system (Hellerstein et al., 2004; Seborg et al., 2004):

- **Inner loop** (synchronous, per-generation): validates each code generation against syntax rules, linting constraints, and file-ownership invariants. Sampling period T_1 (fastest).
- **Middle loop** (asynchronous, per-phase): runs compliance audits, reflexion model checks, and contract verification at phase boundaries. Period $T_2 \gg T_1$.
- **Outer loop** (deliberative, per-milestone): performs ATAM tradeoff analysis (Kazman et al., 2000), strategic alignment review, and ADR lifecycle management. Period $T_3 \gg T_2$.

Let $D(z)$ denote drift introduced by agent-generated code. Each loop has controller $C_k(z)$ and plant $P_k(z)$. The inner loop's closed-loop transfer function is:

$$G_{\text{inner}}(z) = \frac{C_1(z)P_1(z)}{1 + C_1(z)P_1(z)} \tag{8}$$

Each outer loop treats the inner loop's output as its plant.

5.2. Stability

Theorem 5.1 (Cascade Governance Stability)**.** *The three-loop system is stable if and only if:*

1. *Each loop is individually stable (all poles of $G_k(z)$ lie within the unit circle).*
2. *Bandwidth separation holds: $\omega_{\text{bw},1} > \omega_{\text{bw},2} > \omega_{\text{bw},3}$.*
3. *Each outer loop treats the inner as approximately unity gain within its bandwidth.*

Proof. Condition (1) follows from BIBO stability of discrete-time systems. Condition (2) ensures frequency-domain decoupling; without it, outer loop corrections feed into the inner loop's operating range, creating destructive interference. Condition (3) ensures the outer loop's model of the inner loop is accurate within its bandwidth. Under these conditions, the combined characteristic polynomial factors approximately into three independent polynomials. □

Classical cascade control requires a 3:1 to 5:1 bandwidth ratio between adjacent loops. If the inner loop checks every generation (seconds), the middle loop should operate on minutes-to-hours timescales, and the outer loop on days-to-weeks.

5.3. Nyquist-Like Sampling Requirements

Let ω_d be the maximum frequency of architectural drift. Each governance loop must sample at:

$$f_k \geq 2\omega_d^{(k)} \tag{9}$$

where $\omega_d^{(k)}$ is the drift frequency relevant to loop k. If the outer loop samples too slowly, it aliases gradual architectural erosion as apparent stability, missing drift until it becomes catastrophic.

5.4. Governance Stability Conditions

Define architectural coherence $C(t) \in [0, 1]$ as a scalar conformance measure. The dynamics are:

$$\frac{dC}{dt} = \mu \cdot G(t) \cdot (1 - C)^{\beta} - \gamma \cdot R(t) \cdot (1 - C) \tag{10}$$

where $G(t)$ is governance intervention rate, $R(t)$ is agent generation rate, μ is governance effectiveness, γ is agent drift propensity, and $\beta > 1$ captures diminishing governance returns. The asymmetry (β appears in the governance term but not the drift term) reflects the empirical observation that governance faces diminishing returns (easy violations are caught first; subtle ones require disproportionate effort) while drift injection is approximately linear in agent activity (each generation has a roughly constant probability of introducing a violation).

Theorem 5.2 (Critical Governance Capacity)**.** *The system maintains coherence ($dC/dt \geq 0$) if and only if:*

$$G(t) \geq \frac{\gamma}{\mu} \cdot R(t) \tag{11}$$

Governance intervention rate must scale linearly with agent generation rate.

Theorem 5.3 (Governance Bifurcation)**.** *The system undergoes a transcritical bifurcation at $\mu G/\gamma R = 1$:*

- ***Supercritical*** *($\mu G > \gamma R$): a stable interior fixed point at $C^* = 1 - (\gamma R/\mu G)^{1/(\beta-1)} > 0$. The system self-corrects.*
- ***Subcritical*** *($\mu G < \gamma R$): the only stable fixed point is $C^* = 0$ (total coherence collapse).*

Near the bifurcation, recovery time diverges (critical slowing down), and small perturbations in $R(t)$ cause qualitative regime changes.

Remark. *With $\beta > 1$, the equilibrium coherence is always strictly less than 1: perfect conformance is asymptotically unattainable. The gap shrinks as G/R increases but never closes.*

6. The Ratchet

6.1. Lattice of Architectural Constraints

Let $\mathcal{L} = (2^{\mathcal{R}}, \subseteq)$ be the power-set lattice over a universe of governance rules $\mathcal{R}$. A governance state at time t is an element $S_t \in \mathcal{L}$.

Definition 6.1 (Governance Ratchet)**.** *The ratchet operator $\rho : \mathcal{L} \to \mathcal{L}$ is a* closure operator*, satisfying:*

1. ***Extensivity:*** *$S \subseteq \rho(S)$ (rules are only added).*
2. ***Monotonicity:*** *$S \subseteq S' \implies \rho(S) \subseteq \rho(S')$.*
3. ***Idempotency:*** *$\rho(\rho(S)) = \rho(S)$.*

The ADR acceptance workflow is literally a closure operation: accepting a decision adds constraints, those constraints may force additional constraints (transitive closure), and the result is stable under re-application.

Theorem 6.1 (Ratchet Convergence)**.** *Let ρ be a governance ratchet on a finite rule universe $\mathcal{R}$. The sequence $S_0, S_1 = \rho(S_0), S_2 = \rho(S_1), \ldots$ converges to a fixed point $S^* = \rho(S^*)$ in at most $|\mathcal{R}|$ steps.*

Proof. By extensivity, $S_t \subseteq S_{t+1}$, so the sequence is monotonically non-decreasing. Since $\mathcal{R}$ is finite, $\mathcal{L}$ has finite height $|\mathcal{R}|$. A monotone chain of finite height stabilizes. Alternatively, by the Knaster-Tarski theorem (Tarski, 1955), every monotone operator on a complete lattice has a least fixed point. □

6.2. Ossification

Definition 6.2 (Governance Ossification)**.** *A governance state S is* ossified *if* $\text{Allowed}(S) = \{c \in C : c \text{ satisfies all rules in } S\} = \emptyset$.

Theorem 6.2 (Ossification Risk)**.** *If $\mathcal{R}$ contains contradictory rules ($\exists\, r_1, r_2 \in \mathcal{R}$ with no code satisfying both) and the ratchet is purely additive, then any trajectory including both r_1 and r_2 produces an ossified state.*

Proof. By extensivity, once $r_1 \in S_t$ and $r_2 \in S_{t'}$, both persist in all subsequent states. If $r_1 \wedge r_2$ is unsatisfiable, $\text{Allowed}(S) = \emptyset$. □

6.3. Controlled Relaxation

The ossification theorem provides formal justification for relaxation mechanisms. Define a relaxation operator $\delta : \mathcal{L} \to \mathcal{L}$ with $\delta(S) \subseteq S$. The combined governance operator is:

$$\Gamma(S) = \rho(S \setminus \delta(S)) \tag{12}$$

First remove expired or superseded rules via δ, then apply the ratchet closure ρ. Relaxation may be triggered by evidence expiry (a rule's justifying evidence has a validity window), decision supersession (a newer ADR replaces an older one), or explicit deprecation.

Theorem 6.3 (Controlled Descent)**.** *If δ removes only rules whose justifying evidence has expired, and ρ re-derives still-justified consequences, then Γ produces states that are both internally consistent and evidence-backed.*

7. Decision Survival Analysis

7.1. Survival Function

An architectural decision d (an accepted ADR) has a random lifetime T_d. The survival function is $S(t) = P(T_d > t)$, with hazard function $h(t) = -S'(t)/S(t)$. Under exponential decay, $S(t) = e^{-\lambda(t-t_0)}$ with half-life $t_{1/2} = \ln 2/\lambda$.

More realistically, the Weibull distribution gives $h(t) = (k/\lambda)(t/\lambda)^{k-1}$, where $k > 1$ indicates increasing hazard (decisions become more fragile over time) and $k < 1$ indicates infant mortality (Herraiz et al., 2021).

7.2. Competing Risks

Decisions fail for multiple independent reasons (Cox, 1972; Fine and Gray, 1999). Let K index competing risks:

k	Risk Type	Description
1	Technology obsolescence	Chosen technology superseded
2	Requirement change	Business requirements shift
3	Evidence expiry	Justifying data becomes stale
4	Organizational change	Team structure or strategy shifts

The cause-specific hazard for risk k is:

$$h_k(t) = \lim_{\Delta t \to 0} \frac{P(t \leq T_d < t + \Delta t,\ K_d = k \mid T_d \geq t)}{\Delta t} \tag{13}$$

with overall hazard $h(t) = \sum_k h_k(t)$. The cumulative incidence function is:

$$F_k(t) = \int_0^t h_k(u) \exp\left(-\int_0^u h(\tau)\, d\tau\right) du \tag{14}$$

7.3. Empirical Calibration

Based on observed patterns in architectural decay (Le et al., 2018) and technology obsolescence rates, we propose the following parametric hazard models:

Decision Domain	Dominant Risk	Weibull α	Scale λ (mo.)	Median Life
Frontend framework	$k = 1$	1.5–2.5	2–5	1–4 mo.
API contract	$k = 2$	1.2–1.8	5–12	3–9 mo.
Database schema	$k = 3$	1.0–1.4	18–36	12–36 mo.
Security policy	$k = 4$	0.8–1.2	12–24	8–18 mo.

Epistemic status of calibration data. These parameter ranges are *illustrative estimates* derived from practitioner experience, ecosystem churn rates, and the qualitative ordering observed in architectural decay studies (Herraiz et al., 2021; Le et al., 2018). No controlled longitudinal study of decision decay rates exists (§10). The ranges are deliberately wide to reflect this uncertainty. The qualitative ordering (frontend decisions decay fastest; database decisions are most durable) is better supported than the specific numerical values. We present ranges rather than point estimates to avoid false precision.

These calibrations connect to the relaxation operator δ from §6: a decision's evidence expiry window should be proportional to its domain-specific median lifetime.

8. The Theory Preservation Problem

Naur (1985) argues that programs are "alive" when the team possessing the theory remains active and "die" when that team dissolves. When AI generates a substantial fraction of code, a new failure mode appears: *theoretically orphaned implementations*, code for which the theory was never in anyone's head.

8.1. Collins's Taxonomy Applied to Governance

Using Collins (2010)'s taxonomy, we can grade the externalizability of governance-relevant knowledge:

Type	What Governance Can Capture	What Resists Capture
RTK	Decision context, rationale, rejected alternatives, advice received	Obvious-to-the-writer context
STK	Checklists, review heuristics, examples (lossy)	Debugging intuition, code quality "smell"
CTK	(Nothing)	Team norms, "good enough" thresholds

The Dreyfus model (Dreyfus and Dreyfus, 1986) adds a further constraint: expert knowledge is specifically *less* amenable to externalization than novice knowledge. The more expert a practitioner, the more their knowledge resides in pattern recognition that resists decomposition into rules. AI agents operating at the "competent" level of the Dreyfus hierarchy lack the intuitive, holistic perception of proficient and expert practitioners.

8.2. The False Confidence Problem

If Naur (1985) is correct, then any governance artifact (THEORY.md, structured ADRs, extracted decision logs) is a lossy compression. The map-territory problem applies: documentation is a map, not the territory. The risk: a team possessing documentation may believe it understands the system's theory when it actually possesses only a compressed, lossy representation. This could be worse than having no documentation, because:

- No documentation produces appropriate humility ("we don't know why this is here").
- Lossy documentation produces false confidence ("THEORY.md says it's for X, so we'll change it accordingly").

However, this argument has practical limits (§11). Lossy compression is still enormously useful (JPEG images demonstrate this). The question is not whether documentation is perfect but whether it is better than nothing, and whether its limitations are understood.

9. Related Work

Architecture Decision Records. Nygard (2011) proposed the lightweight ADR format. The Architecture Advice Process (Harmel-Law, 2023) adds a governance layer: anyone can make architectural decisions, but must first consult affected parties and domain experts. This is not an approval process; the decision-maker is "in no way obliged to agree with the advice."

Fitness functions. Ford et al. (2017) define an architectural fitness function as "an objective integrity assessment of some architectural characteristic(s)." Tools like ArchUnit encode these as executable tests. The gap is in ADR-to-fitness-function automation, which tools like Archgate (Archgate, 2025) are beginning to address through the ratchet model.

Reflexion models. Murphy et al. (1995, 2001) provide the formal foundation for conformance checking. Extensions include behavioral reflexion models for state machines and UML-based variants. The SEI developed a CI-integrated prototype detecting nonconformances "within minutes, instead of the months or years it takes today."

Agent Decision Records. The AgDR format (me2resh, 2025) extends ADRs with required metadata (agent name, model ID, session, trigger type), addressing traceability gaps when AI agents make architectural choices.

Control theory for software. Hellerstein et al. (2004) provided the foundational treatment of feedback control for computing systems. Our contribution is applying cascade control specifically to multi-granularity governance, with stability and sampling analysis.

Conway's Law. Conway (1968) established the homomorphism between organizational and system structure. Nagappan et al. (2008) found that organizational metrics predict failure-proneness with ~85% precision and recall, *outperforming all code metrics*. MacCormack et al. (2008) validated the "mirroring hypothesis" empirically. Skelton and Pais (2019) operationalized the inverse Conway maneuver through Team Topologies.

10. Empirical Calibration and Gaps

The formal framework requires calibration data. We assess what exists honestly:

Data Category	Quality	Key Gap
Code duplication (AI era)	High	211M lines, 5-year longitudinal (GitClear, 2025)
Refactoring decline	High	Same dataset; causal attribution unclear
Delivery stability	High	DORA survey (Google Cloud, 2025); self-reported
ADR compliance rates	Very low	No quantitative studies exist
Decision decay rates	Very low	Qualitative only
Drift velocity	Low	Tools exist; no cross-project baseline
Governance effectiveness	Low	Case studies only; no controlled trials

The most important finding is the gap between the confidence with which governance practices are recommended and the empirical evidence supporting them. Conway's Law is well-validated (Conway, 1968; MacCormack et al., 2008; Nagappan et al., 2008). The specific governance mechanisms proposed to work within its constraints are supported primarily by practitioner experience and theoretical argument, not controlled empirical studies. This does not make them wrong; it makes them *unvalidated*, a different epistemic status than either "proven" or "disproven."

The measurement problem. Goodhart's Law (Goodhart, 1975) applies: when a governance metric becomes a target, it ceases to be a good measure. Coupling targets incentivize restructuring that reduces measured coupling while introducing unmeasured coupling through shared configuration. Coverage targets incentivize trivial fitness functions. The defense is metric rotation, metric triangulation, and cultural alignment.

Uncomputability. "True" architectural quality is undecidable: fitness for purpose requires knowing future requirements; optimal decomposition is NP-hard; semantic coupling is undetectable by static analysis. Practical governance metrics are always approximations. The question is whether their failure modes are understood and tolerable.

11. Engaging Contrarian Positions

We present five contrarian positions in steelmanned form and evaluate what the evidence actually supports.

11.1. "Governance Is Overhead"

The strongest version: governance has a cost curve intersecting its benefit curve at a specific scale, and below that scale the cost exceeds the benefit. For a two-person startup, time spent writing ADRs is time not spent shipping features.

What the evidence shows: Governance cost is roughly fixed per mechanism; benefit scales with team size and codebase complexity. The crossover appears at 5–10 developers, or more precisely, at the point where no single person holds the complete mental model. For distributed teams, formal governance becomes valuable earlier because informal channels (hallway conversations) are degraded or absent.

11.2. "ADRs Are Documentation Theater"

ADRs can drift from purpose, enable responsibility diffusion, and suffer the staleness problem (Spotify Engineering, 2020). Despite recommendations from ThoughtWorks, AWS, Microsoft, and Google, **no published empirical study demonstrates that ADRs improve measurable software quality outcomes** (defect rates, time-to-market, maintenance cost). The evidence is entirely practitioner reports and observational accounts.

What the evidence shows: ADRs provide clear value for onboarding and cross-team alignment. The primary failure mode is absent content (decisions made without ADRs) or bloated content (too many decisions documented), not wrong content.

11.3. "Naur Is Right and the Problem Is Unsolvable"

If theory cannot be externalized, THEORY.md is lossy compression creating false confidence. Acting on extracted "decisions" may be worse than no documentation.

What the evidence shows: Naur wrote in 1985 about small teams with long tenure. Modern software involves high turnover across time zones. The alternative to lossy documentation is not "direct contact with theory-holders" but *no knowledge transfer at all*. Lossy compression is still useful (JPEG). Documentation with explicit uncertainty markers ("confidence: medium," "last validated: 2025-03") addresses the false confidence risk.

11.4. "Conway's Law Makes Governance Futile"

If architecture mirrors org structure (Conway, 1968), and org structure predicts quality better than code metrics (Nagappan et al., 2008), then code-level governance treats a symptom.

What the evidence shows: Conway's Law is "powerful enough that you're doomed to defeat if you try to fight it." But it admits an inverse: the inverse Conway maneuver (Skelton and Pais, 2019) deliberately restructures teams to produce desired architecture. Code-level governance and organizational governance are complementary, not substitutes. The novel question for AI agents: if agents mirror the org structures of the teams that build them, then agent governance requires organizational awareness.

11.5. "The Ratchet Is Too Rigid"

Monotonically increasing strictness prevents legitimate architectural evolution. The "greenfield escape" pattern: the only way to evolve is to start a new project.

What the evidence shows: The tension is real but the strongest governance systems already incorporate relaxation mechanisms (§6). Fitness functions can be deprecated (Ford et al., 2017). Evidence expiry provides a natural clock for rule relaxation. The ratchet critique applies most forcefully to systems that treat rules as permanent rather than contextual, which is precisely what the lattice descent operator δ addresses.

12. Design Principles

The formal framework yields the following actionable principles for governance architecture:

1. **Respect the capacity bound.** If drift rate exceeds governance channel capacity, increasing governance sophistication is futile. The only remedy is increasing capacity (automating governance layers) or decreasing drift rate (improving agent prompting, injecting architectural context).
2. **Govern at three granularities.** Synchronous (per-generation) checks prevent individual violations. Asynchronous (per-phase) audits catch emergent drift. Deliberative (per-milestone) reviews address strategic alignment. All three are necessary; each alone is insufficient.
3. **Maintain bandwidth separation.** Adjacent governance loops should differ by at least 3:1 in sampling rate. Violating this produces destructive interference (over-aggressive inner loops that create oscillation, or outer loops that alias drift as stability).
4. **Provision above the bifurcation.** Organizations should maintain governance capacity with margin above the critical ratio $\mu G/\gamma R = 1$. Operating near the bifurcation point is fragile: a temporary spike in agent activity can push the system into coherence collapse with hysteresis.
5. **Close the ratchet with evidence expiry.** Every rule should have an evidence validity window. The relaxation operator δ prevents ossification while preserving still-valid constraints.
6. **Accept residual theory loss.** Perfect externalization is impossible (Conjecture 3.1). Design governance to tolerate a non-zero residual, using it as a risk measure. Supplement artifact-based governance with social mechanisms (code review conversations, architecture advice processes) for knowledge that cannot be externalized.
7. **Make logs rich.** The double bottleneck (Theorem 3.3) shows that governance fidelity is bounded by log richness. Agent logs must externalize decisions, assumptions, and rejected alternatives, not just final code.
8. **Track decision age, not just decision content.** Survival analysis (§7) shows that decision validity decays with domain-specific half-lives. Governance should proactively surface decisions approaching their expected half-life.
9. **Measure governance, not just architecture.** Monitor governance response time, escalation frequency, and the ratio $\mu G/\gamma R$. Architectural metrics alone cannot detect governance failure; they can only detect its consequences after the fact.
10. **Acknowledge the empirical gap.** The strongest evidence is that the *problem* is real (AI accelerates quality degradation). The evidence that *governance solves it* remains largely anecdotal. Design governance systems to be testable: track drift trajectories and decision decay so that future work can validate the framework.

13. Conclusion

We have presented a formal framework for governance architecture in AI coding harnesses, grounding it in information theory, control theory, survival analysis, and lattice theory. The central result, the governance capacity bound (Theorem 4.1), establishes that no governance scheme can prevent unbounded architectural drift when the drift rate exceeds governance channel capacity. This is not a statement about governance quality; it is a statement about information-theoretic limits.

The cascade control model shows that multi-granularity governance is not merely a good practice but a necessary architectural pattern, with formal stability conditions and Nyquist-like sampling requirements. The governance bifurcation (Theorem 5.3) identifies a critical ratio separating self-correcting from collapsing regimes, with practical implications for capacity provisioning.

The ratchet formalization reveals both its power (guaranteed convergence, monotone enforcement) and its pathology (ossification under contradictory rules), motivating the controlled relaxation operator. The survival analysis connects decision lifecycle management to the relaxation mechanism through empirically calibrated decay rates.

Three limitations deserve emphasis. First, the information-theoretic analogy is rigorous in structure but the "channel" and "capacity" of governance are not directly measurable in the way Shannon's quantities are for communication channels. The framework provides qualitative insight and design guidance rather than computable bounds. Second, the empirical calibration data is sparse: decision decay rates and governance effectiveness remain largely unmeasured. Third, the tacit divergence (Conjecture 3.1) remains a conjecture; formalizing it fully would require a theory of tacit knowledge that connects Polanyi's philosophical claims to measurable information-theoretic quantities, which is an open problem.

The framework's honest contribution is in structuring the governance problem formally. Architectural drift is the integral of the trajectory-snapshot gap over time. Evidence has a half-life. The ratchet converges but risks ossification. Governance channel capacity is finite. These are not slogans; they are formal claims with defined conditions, identified limitations, and (where possible) empirical calibration. The gap between these formal results and validated practice is itself the most important finding: governance architecture as a field has sophisticated theory and almost no empirical validation.

Acknowledgments

This paper draws on the governance pillar analysis of the HTX Harness Engineering project. The formal framework synthesizes techniques from information theory, control theory, survival analysis, and the philosophy of tacit knowledge into a unified treatment of architectural governance under AI acceleration.

References

Archgate. Archgate: Executable ADRs for AI governance. https://github.com/archgate/cli, 2025.

G. J. Chaitin. Information-theoretic limitations of formal systems. *Journal of the ACM*, 21(3):403–424, 1974.

H. Collins. *Tacit and Explicit Knowledge*. University of Chicago Press, Chicago, 2010.

M. E. Conway. How do committees invent? *Datamation*, 14(4):28–31, 1968.

T. M. Cover and J. A. Thomas. *Elements of Information Theory*. Wiley-Interscience, 2nd edition, 2006.

D. R. Cox. Regression models and life-tables. *Journal of the Royal Statistical Society: Series B*, 34(2): 187–220, 1972.

H. L. Dreyfus and S. E. Dreyfus. *Mind Over Machine: The Power of Human Intuition and Expertise in the Era of the Computer*. Free Press, New York, 1986.

J. P. Fine and R. J. Gray. A proportional hazards model for the subdistribution of a competing risk. *Journal of the American Statistical Association*, 94(446):496–509, 1999.

N. Ford, R. Parsons, and P. Kua. *Building Evolutionary Architectures: Support Constant Change*. O'Reilly Media, 2017.

GitClear. AI copilot code quality research 2025. https://www.gitclear.com/ai_assistant_code_quality_2025_research, 2025.

C. A. E. Goodhart. Problems of monetary management: The UK experience. *Papers in Monetary Economics*, 1975.

Google Cloud. 2025 DORA report. https://cloud.google.com/blog/products/ai-machine-learning/announcing-the-2025-dora-report, 2025.

A. Harmel-Law. *Facilitating Software Architecture: Empowering Teams to Make Architectural Decisions*. O'Reilly Media, 2023.

J. L. Hellerstein, Y. Diao, S. Parekh, and D. M. Tilbury. *Feedback Control of Computing Systems*. Wiley-IEEE Press, 2004.

I. Herraiz et al. Software evolution: The lifetime of fine-grained elements. *PeerJ Computer Science*, 7:e372, 2021.

R. Kazman, M. Klein, and P. Clements. ATAM: Method for architecture evaluation. Technical Report CMU/SEI-2000-TR-004, Carnegie Mellon University Software Engineering Institute, 2000.

D. M. Le, C. Carrillo, R. Capilla, and N. Medvidovic. An empirical study of architectural decay in open-source software. In *IEEE International Conference on Software Architecture (ICSA)*, 2018.

A. MacCormack, J. Rusnak, and C. Y. Baldwin. Exploring the duality between product and organizational architectures. *Harvard Business School Working Paper*, (08-039), 2008.

me2resh. Agent decision records. https://github.com/me2resh/agent-decision-record, 2025.

G. C. Murphy, D. Notkin, and K. Sullivan. Software reflexion models: Bridging the gap between source and high-level models. In *Proceedings of the 3rd ACM SIGSOFT Symposium on Foundations of Software Engineering*, pages 18–28, 1995.

G. C. Murphy, D. Notkin, and K. Sullivan. Software reflexion models: Bridging the gap between design and implementation. *IEEE Transactions on Software Engineering*, 27(4):364–380, 2001.

N. Nagappan, B. Murphy, and V. Basili. The influence of organizational structure on software quality: An empirical case study. In *Proceedings of the 30th International Conference on Software Engineering*, pages 521–530. ACM, 2008.

P. Naur. Programming as theory building. *Microprocessing and Microprogramming*, 15(5):253–261, 1985.

I. Nonaka and H. Takeuchi. *The Knowledge-Creating Company*. Oxford University Press, 1995.

M. Nygard. Documenting architecture decisions. Cognitect Blog, November 2011.

M. Polanyi. *The Tacit Dimension*. Doubleday, New York, 1966.

G. Ryle. *The Concept of Mind*. Hutchinson, London, 1949.

D. E. Seborg, T. F. Edgar, and D. A. Mellichamp. *Process Dynamics and Control*. Wiley, 2004.

C. E. Shannon. A mathematical theory of communication. *Bell System Technical Journal*, 27(3): 379–423, 1948.

C. E. Shannon. Coding theorems for a discrete source with a fidelity criterion. In *IRE National Convention Record*, volume 7, pages 142–163, 1959.

M. Skelton and M. Pais. *Team Topologies: Organizing Business and Technology Teams for Fast Flow*. IT Revolution Press, 2019.

Spotify Engineering. When should i write an architecture decision record? https://engineering.atspotify.com/2020/04/when-should-i-write-an-architecture-decision-record, April 2020.

Sylvester. The architecture is the plan: Fixing agent context drift, 2026.

A. Tarski. A lattice-theoretical fixpoint theorem and its applications. *Pacific Journal of Mathematics*, 5(2):285–309, 1955.

H. Tsoukas. Do we really understand tacit knowledge? In *The Blackwell Handbook of Organizational Learning and Knowledge Management*, pages 410–427. Blackwell, 2003.

A. Governance Information Budget: Full Statement

Theorem A.1 (Governance Information Budget)**.** *A governance system for AI-accelerated development operates under four simultaneous constraints:*

1. ***Extraction constraint:***

$$I(T;G) \leq I(T;L) \leq H(T) - R_{\text{tacit}}^{-1}(\textit{log budget}) \tag{15}$$

   where the last term reflects that tacit knowledge components cannot be fully captured regardless of log length.
2. ***Drift constraint:*** *Stability requires $\mu\bar{c} \geq \lambda\bar{\delta}$; correction rate must match drift injection rate.*
3. ***Capacity constraint:*** *Coherence requires $R_{\text{drift}} < C_{\text{gov}} = \sum_{\ell} C_{\ell}$; drift rate must remain below governance channel capacity.*
4. ***Fidelity constraint:*** *$I(T;G) \geq F_{\min}$ for some minimum governance fidelity; governance actions must carry enough information about the theory.*

These constraints are in tension: increasing agent velocity λ increases R_{drift} (constraint 3) and the volume of logs requiring extraction (constraint 4), while the human component of C_{gov} remains fixed and $R_{\text{tacit}}(D)$ diverges (constraint 1).

B. Notation Summary

Symbol	Meaning
T	Program theory (mental model)
$\hat{T}$	Reconstructed theory from artifacts
$d(t, \hat{t})$	Distortion between original and reconstructed theory
$R(D)$	Rate-distortion function
$K(t)$	Kolmogorov complexity of theory instance t
P	Intended architecture distribution
Q_t	Actual architecture distribution at time t
$\Delta(t)$	Architectural drift (KL divergence)
λ	Agent modification rate
μ	Governance effectiveness parameter
γ	Agent drift propensity
$\bar{\delta}$	Expected drift per modification
$\bar{c}$	Expected drift reduction per correction
C_{gov}	Governance channel capacity
R_{drift}	Drift rate (bits per unit time)
L	Executor log
(D, A, R)	Structured extraction (decisions, assumptions, rejected alternatives)
G	Governance actions
S_t	Governance state (rule set) at time t
ρ	Ratchet (closure) operator
δ	Relaxation operator
Γ	Combined governance operator (ρ after δ)
$h_k(t)$	Cause-specific hazard for risk k
$F_k(t)$	Cumulative incidence function for risk k
$C(t)$	Architectural coherence $\in [0, 1]$
β	Governance diminishing returns exponent
$F_{\min}$	Minimum governance fidelity

C. Empirical Data Sources

Source	Finding	Methodology	Confidence
GitClear 2025	8× duplication increase; refactoring ↓60%	211M lines, 5-year longitudinal	High
DORA 2025	Individual tasks ↑ 21%; org throughput flat; ↓ stability	Large survey	High
Nagappan et al.	Org metrics predict bugs at 85%	Windows Vista case study	High
Le et al. 2018	Constant architectural decay rates	Open-source longitudinal	Medium
Herraiz et al. 2021	Infant mortality in code elements	PeerJ, fine-grained analysis	Medium